

B 配套实验与开源代码

本书配套设计了四组课程实验，仓库地址为 <https://gitee.com/hnu-cloudcomputing/cloud-compute-book-code>，其中包含程序骨架、实验说明和自动评分工具。四组实验都使用 Go 语言编写，围绕同一个文字游戏逐步扩大系统规模，读者不需要从零搭建项目，只需在预留的 TODO 位置填入核心逻辑，再通过自动化脚本验证得分。

每组实验大致对应书中一章或两章的内容：实验一对应第 1 章的网络通信与客户端/服务器架构；实验二对应第 2 章的并发与共享状态；实验三对应第 3 章和第 4 章的分布式协同与一致性；实验四对应第 5 章和第 6 章的容器编排与弹性伸缩。读者可以在学完对应章节后进入对应实验，把正文中的机制落到可运行的程序上，再通过观察运行结果反过来加深对概念的理解。

仓库中 Lab1 到 Lab4 四个目录分别对应四组实验，每个目录下都有三个部分：README.md 给出实验目标、任务分解和评分标准；student/ 是学生填写目录，大部分代码已经写好，只在关键位置留有 TODO；test/ 提供自动测试脚本，运行后会输出各用例的通过情况和对应得分。

四组实验的推进线索是同一个游戏系统的规模扩张。实验一先建立两个客户端与一个服务器之间的网络闭环；实验二把双人对战扩展为多人实时世界，处理同一进程内的并发访问；实验三进一步拆出多张地图和多个逻辑节点，引入跨节点路由、事务与故障恢复；实验四把这套分布式游戏部署到 Kubernetes 集群，在正确性、可用性、弹性和资源成本之间反复调优。它们共同展示了一个在线系统从能够通信，到能够并发运行、跨节点协作并在云平台上持续服务的演进过程。

B.1 实验一：双人对战的网络闭环

实验一构造一个最小但完整的客户端/服务器系统。两个玩家分别运行客户端，通过 TCP 连接同一个游戏服务器。客户端负责读取玩家的移动、攻击和使用药水等操作，并把这些操作编码成消息发送给服务器；服务器为玩家分配身份，维护坐标、生命值和回合状态，检查移动是否越界、攻击距离是否合法，再把更新后的状态发送给双方。

这个实验包含三部分内容。第一部分是 TCP 连接的建立：服务器监听端口并接受连接，客户端主动连接服务器。第二部分是应用层消息传输：程序使用 JSON 表示消息，并在一条持续存在的字节流上依次编码和解析多条消息。第三部分是服务器权威裁决：客户端提交的是“向哪个方向移动”或“发起攻击”这样的操作意图，真正的坐标变化、伤害计算和胜负判断都由服务器完成。

完成后的程序能够让两个客户端通过服务器进行一场完整对战。读者可以由此观察到，网络连通只解决了字节如何到达；在线系统还必须定义消息怎样区分、状态由谁保存以及结果由谁决定。

实验一以填空形式呈现，学生需要在 TCP 监听与连接、JSON 消息收发、移动与攻击判定三个位置填入核心代码。测试脚本会自动启动服务器和两个客户端，依次验证连接握手、边界检查、攻击距离和断线处理等用例，读者可以根据失败定位快速回溯具体机制。后续实验将在这个最小闭环上继续增加并发和分布式协作能力。

B.2 实验二：多人世界中的并发控制

实验二把双人回合制对战扩展为多人实时开放世界。玩家可以随时加入或离开，服务器为每个连接启动独立的 Goroutine，并周期性地向所有在线玩家广播世界快照。客户端同样需要并发处理：主执行流阻塞在键盘输入上，另一个 Goroutine 负责接收服务器推送，两者共同维护屏幕上的角色位置与事件记录。只有这样，玩家才能在操作自己角色的同时，实时看到其他玩家的移动、攻击、死亡与复活。

多人同时活动后，所有连接处理逻辑都会访问同一份世界状态。实验围绕这份共享状态设计并发安全接口：加入玩家、离开世界、移动和攻击会修改玩家集合，需要互斥写入；生成广播快照只读取状态，可以使用读锁允许多个读取同时进行。玩家死亡后的延迟复活还会启动新的 Goroutine，因此程序必须明确当前操作何时释放写锁，保证复活逻辑随后能够重新取得锁，避免后台任务长期等待甚至形成死锁。

这个实验呈现了“功能能够运行”和“并发执行仍然正确”的区别。程序可能在少量玩家下表现正常，却在多个连接同时读写映射时出现数据竞争、状态覆盖或死锁。通过多人交互、周期广播和 Go 内置的 race detector 竞争检测，读者可以观察 Goroutine 如何组织并行工作，也可以理解为什么共享状态必须通过明确的临界区和并发安全接口加以保护。

B.3 实验三：多地图与多节点协同

实验三把单地图、单节点的世界扩展为由三张地图和三个逻辑服务节点组成的分布式战场。每张地图独立维护玩家、NPC、宝物和地图版本，并拥有负责读写的主节点与保存副本的节点。玩家可以在地图之间切换，系统必须把操作路由到当前地图的主节点，同时维护玩家会话中的地图和节点位置。

地图放置与节点变化构成了实验的第一条主线。系统使用一致性哈希为地图选择主节点和副本节点，并在成员变化时生成迁移计划；Gossip 成员表传播节点的存活、可疑和失效状态。当某个节点失效后，系统需要提升仍然可用的副本，修正在线玩家的路由，并把新的地图归属写入 Raft 元数据日志。归属变更只有在多数节点复制后才能提交，从而避免不同节点同时把自己视为主节点。

跨节点操作构成了第二条主线。不同地图上的玩家转移战利品时，转出与转入必须同时成功或同时撤销，实验用两阶段提交组织这一过程。世界 Boss 的生命值则由所有地图共同观察和修改，用来呈现跨分片共享状态。与此同时，账号资料与在线会话分别作为冷数据和热数据保存，地图检查点会写入存储并同步到副本，为节点故障后的状态恢复提供起点。

这些机制不是彼此独立的算法练习。一次节点故障会同时影响成员状态、地图归属、请求路由和状态恢复；一次跨节点交易也会同时涉及玩家位置、参与节点和提交结果。实验还提供了一个独立的管理命令行工具，读者可以用它查看集群拓扑、手动模拟节点故障，以及在故障后观察系统如何自动完成副本提升和路由修正。实验三的重点正是把放置、通信、事务、一致性和恢复放进同一条游戏流程，观察分布式机制如何共同维持一个可继续运行的世界。

B.4 实验四：Kubernetes 上的部署与弹性运行

实验四把分布式文字游戏部署到自行配置的 Kubernetes 集群。与前三组不同，这个实验的重点不是再增加新的功能，而是把一个已经写好的分布式系统搬到云平台上，让读者在真实的容器编排环境中体会弹性伸缩、故障恢复和资源成本之间的取舍。

待部署的系统由一个对外提供 TCP 入口的 gateway、一个负责登录与全局协调的 coordinator、green、cave、ruins 三个地图服务以及保存共享状态的 Redis 组成。客户端请求从集群外进入 gateway，再经过 coordinator 到达相应地图服务；玩家资料、在线会话、地图检查点和服务租约等状态保存在 Redis 中。

实验的第一层任务是把系统跑起来。读者需要把不同程序制作成容器镜像，再用 Kubernetes 对象描述整套服务。五个业务组件分别使用 Deployment 管理副本，Redis 使用 StatefulSet 保存状态；Service 为外部入口和集群内部组件提供稳定访问地址。资源声明、健康检查和配置注入共同决定 Pod 能否被调度、何时可以接收流量，以及组件之间能否按预期连接。实验还涉及 ServiceAccount 与 RBAC，使业务组件能够在最小权限范围内读取或更新必要的集群信息。完成这一层后，整套服务能够在集群中正常运行，客户端可以通过固定地址接入游戏。

但”跑起来”只是实验的起点。实验真正的重点，是让读者在弹性、可用性和资源成本之间反复调优，观察不同配置带来的系统行为差异。gateway、coordinator 与三个地图服务都配置水平自动伸缩，根据负载增加或减少副本，而 Redis 作为状态中心不参与同样的水平伸缩。业务 Pod 在被缩容、滚动更新或手动删除时，需要先进入排空状态，通过就绪检查退出流量路径，并尽量完成状态保存和存量操作。系统还可以根据实例承载的活跃玩家数量调整 Pod 的删除代价，降低平台优先回收繁忙实例的概率。这些配置没有标准答案——CPU 目标值设得高，资源利用率好但扩容不及时；设得低，响应快但副本数容易虚高。读者需要自己权衡并验证。

配套的竞技评分从六个维度衡量部署质量：空闲时是否发生无意义扩容、低压力下是否保持稳定、高压力下能否及时增加容量、扩容和重试后状态是否一致、Pod 被删除后服务能否恢复，以及资源申请和副本数量是否合理。评分不按绝对性能排序，而是看系统在该有的时候有、不该有的时候不浪费。因此，实验四并不以”成功创建一组 Pod”为终点，而是要求读者用运行结果说明，自己的部署策略在正确性、可用性、弹性和资源成本之间做出了怎样的平衡，以及为什么这种平衡是合理的。